



Cats vs. Dogs: Image Classification Using a Custom ResNet

Daniel Onyedikachi Eta

ID: 43092028

Keli Kobla Kemeh

ID: 48062028

Michael Kwadwo Obour

ID: 67112028

Department of Computer Science & Information Systems, Ashesi University

CS452: Machine Learning

Dr. James Okae

August 21, 2026

Introduction

Computer Vision, as a field, has brought with it the exciting and emerging possibility of using automated systems to make important and referential classification decisions that have, for so long, relied on expert human actors. Because of this, the conceptualisation, development, and implementation of neural networks that accurately identify features within image pixels have compounding effects on the efficacy of consumer applications that rely on image detection and categorisation to make important decisions at scale. For systems that can adequately classify animals, as our project does, this technology can be particularly leveraged in fields like veterinary diagnostics and pet care, where intelligent systems can, for instance, classify pictures of animals under correspondingly specialised veterinarians for consultation.

The existing literature for Convolutional Neural Networks (CNNs) is extensive; it has been a revelatory technology in allowing our systems to identify spatial patterns within input data and thus classify objects. However, standard CNNs frequently suffer from vanishing gradients when the depth of network layers increases. They also overfit rapidly on small-to-medium datasets when regularisation techniques are not robust, leading to poor classification performances.

Thus, the objective of this project is to construct an accurate, end-to-end, and lightweight computer vision pipeline using PyTorch. We intend to circumvent the aforementioned issue—of poor classification performance on deeper CNN architecture—by adopting a conventional (but custom) Residual Network (CatDogResNet) that balances capacity and compute efficiency without relying on pre-trained backbones. We will train and rigorously benchmark the middle using metrics on accuracy, recall, F1-score, and a confusion matrix to finally evaluate a test set. The goal and significance of this is generally to demonstrate the subtle power of simple residual-skip connections and adaptive spatial pooling in enabling deep feature extraction, even without billions of parameters.

System Design & Methodology

Our experiment began, as model development always does, with a dataset. After experimenting with a couple, we settled on Microsoft’s popular “Cats Vs. Dogs” dataset that we retrieved via the Hugging Face Hub. It is a sizeable dataset consisting of circa 20000 raw images.

Because this dataset came externally from HuggingFace, we spent a code block in our Jupyter Notebook implementing a `CatsDogsDataset(Dataset)` adapter class that essentially sanitises the arbitrary input formats (like converting RGBA/grayscale into the uniform 3-channel RGB) to bridge Hugging Face’s standard dictionary objects into PyTorch tensors on the fly for the data loaders to work with. From here, we then partition the dataset into splits for training,

validation, and testing to eliminate data leakage: 70% for training, 15% for validation, and 15% for testing. We also augment the training partition of the dataset only to artificially expand the dataset variety and prevent the network from memorising training samples. There is no need for augmentation in the validation or testing datasets; we wanted those partitions to serve as uncorrupted, stable, and standard benchmarks.

Model Architecture

The model architecture is fairly simple. Rather than relying solely on sequential convolutional layers, the network uses a custom `ResidualBlock` building block. In a standard deep network, backpropagating gradients can vanish as network depth increases. We address this by formulating the layer mapping as a residual learning problem:

$$H(x) = \text{ReLU}(F(x) + x)$$

where x represents the input activation tensor and $F(x)$ denotes the residual mapping learned by two sequential 3×3 convolutions with batch normalisation. The identity shortcut connection ($+x$) creates an unobstructed highway for gradient flow, ensuring stable parameter updates throughout training.

Funneling Features Hierarchically

The network processes input images of size $3 \times 128 \times 128$ through a three-stage resolution funnel:

1. **Stem Stage:** An initial 3×3 convolution extracts 32 feature channels, followed by max pooling to halve the spatial resolution ($128 \times 128 \rightarrow 64 \times 64$).
2. **Residual Stages & Downsampling:** The architecture passes intermediate activations through three residual blocks interleaved with downsampling layers. At each transition, max pooling reduces the spatial dimensions ($64 \rightarrow 32 \rightarrow 16$) while convolutions systematically double channel capacity ($32 \rightarrow 64 \rightarrow 128$). This enables early layers to capture low-level textural primitives while deeper layers learn abstract anatomical features (e.g., ear shape and muzzle geometry).

Standard convolutional networks typically flatten high-dimensional spatial feature maps directly into dense linear layers, incurring millions of parameters and severe overfitting. To prevent this, our architecture introduces global average pooling (`AdaptiveAvgPool2d((1, 1))`). This operation averages each 16×16 feature map into a single representative scalar, compressing the tensor from $(128, 16, 16)$ down to $(128, 1, 1)$ prior to flattening. The resulting 128-dimensional embedding is routed through a compact multi-layer perceptron with dropout ($p = 0.3$) to output the final two-class logits.

Experiments and Results

To maximise GPU compute utilisation and eliminate I/O bottlenecks, we implemented the data loading asynchronously using PyTorch’s DataLoader with pinned host memory (`pin_memory = True`), paired with non-blocking VRAM transfers (`to_device(..., non_blocking = True)`).

The optimisation process of the model used Multi-class Cross-Entropy Loss ($L = -\sum y_c \log(\hat{y}_c)$) as the objective function, the popular Adam optimiser to handle the gradient descent (with a base learning rate of $\alpha = 10^{-3}$), and the ReduceLROnPlateau scheduler to dynamically alter the learning rate when optimisation progress stalled. The batch size during this experiment was 128 for training, and because the validation/testing phases only compute a forward pass, we increased their batch size to 256.

To safeguard against overfitting, we also implemented a dropout of 0.3, so that the model would randomly turn off 30% of its neurons during each batch. After each epoch, the model also used a separate validation set to measure how well it was generalising to other images.

We also implemented a type of “model checkpointing” in the system. Essentially, whenever the model scores its lowest error (validation loss) so far, it saves a copy of its brain (the weights) to a file named `catdog_best_model.pth`. Even if the model begins to overfit or perform worse in later epochs, it reloads that best saved version before running the final test.

Quantitative Evaluation

The final model was evaluated on the completely unseen held-out test split ($N = 3,344$). As summarised in Table 1, the network achieved an overall classification accuracy of **88.13%**.

Class	Precision	Recall	F1-Score	Support
Cat	0.8407	0.9362	0.8859	1,646
Dog	0.9305	0.8280	0.8763	1,698
Accuracy			0.8813	3,344
Macro Avg	0.8856	0.8821	0.8811	3,344
Weighted Avg	0.8863	0.8813	0.8810	3,344

Table 1: Performance metrics on the held-out test set.

Results Discussion & Error Analysis

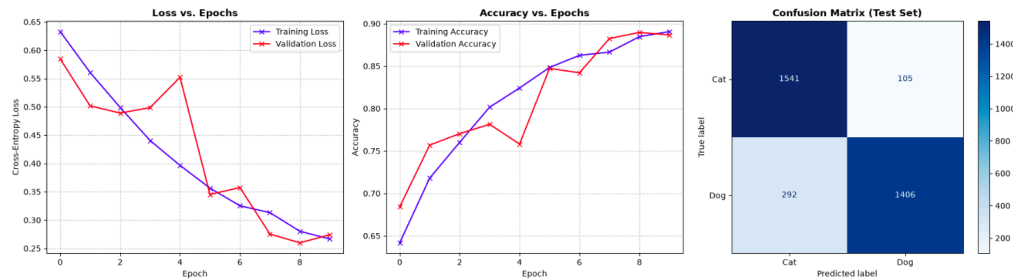


Figure 1: Training and validation dynamics across 10 epochs alongside the final test confusion matrix.

The diagnostic dashboard (Figure 1) reveals findings regarding convergence dynamics and class-conditional error rates:

1. **Convergence & Generalization Trajectory:** As illustrated in the loss and accuracy curves, training loss dropped monotonically from 0.632 to 0.267. The validation loss stabilised near 0.274, tracking training loss closely without divergence. This lack of an overfitting gap validates the efficacy of the geometric data augmentations and dropout regularisation.
2. **Mid-Training Optimisation Volatility:** A transient spike in validation loss occurred at Epoch 4 (rising from 0.490 to 0.552, accompanied by a drop in validation accuracy to 75.8%) before rapidly recovering by Epoch 5. This oscillation indicates a temporary descent into a steep gradient valley before the adaptive optimiser stabilised the trajectory.
3. **Error Asymmetry and Morphological Bias:** The confusion matrix reveals an asymmetric error distribution:
 - True Cats misclassified as Dogs: **105** (6.38% false negative rate).
 - True Dogs misclassified as Cats: **292** (17.20% false negative rate).

While the model exhibits strong precision when predicting dogs (93.05%), it demonstrates a higher recall for cats (93.62%). We hypothesise that this cat-prediction bias stems from anatomical variance: feline facial geometry across breeds remains relatively uniform (e.g., pointed ear structures, whisker pads), whereas canine breeds exhibit extreme morphological variance in snout length, ear posture, fur texture, and body size.